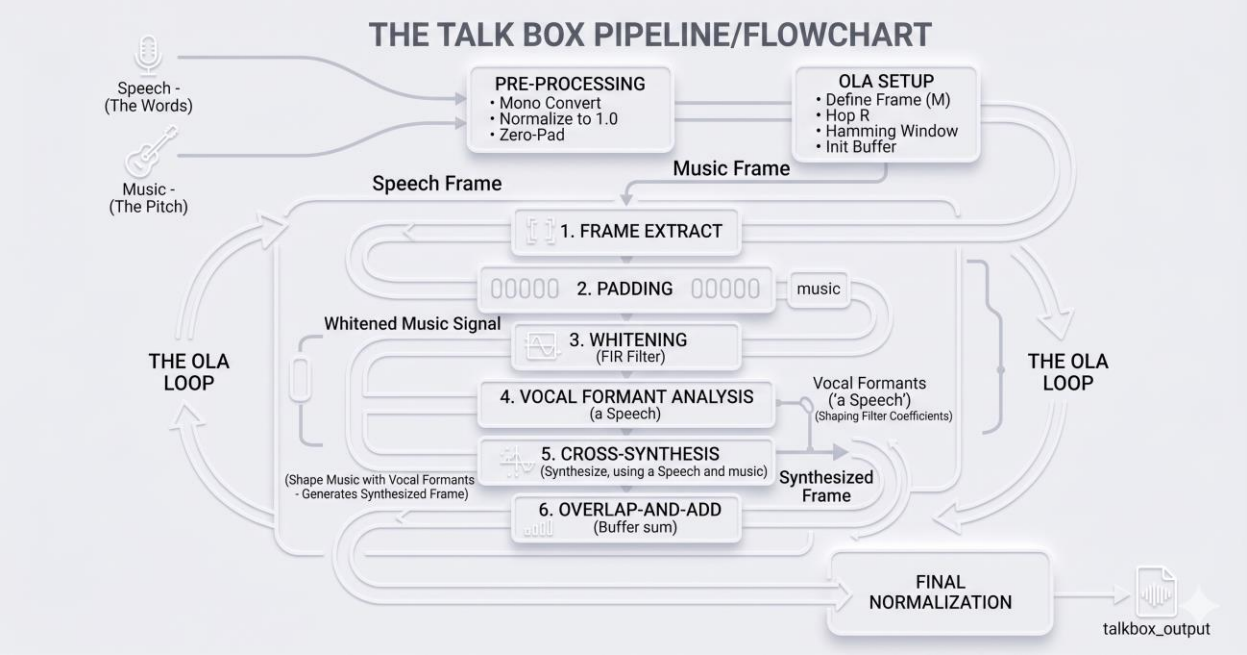
SASP (DAAB) - Homework 1

Surname	Henry	Person Code	11136207	Email	sherifhenry.henry@mail.polimi.it

1.0 System Overview and Steps Explanation:

The goal of this Homework is to emulate the talk-box effect. In order to achieve that, I followed the algorithm shown, which is heavily based on the LPC lecture (specially the last slide talking about the cross-synthesis concept) and the OLA lecture, in this system flow chart:



- Firstly, I opened the 2 files using audio read to get back the samples and the Fs then I ensured that the 2 files have the same sampling frequency with a small check and an error message.
- Then afterwards I ensured the 2 signals are in mono by taking the mean of the 2 channels as the Levinson function expect 1D mono input and give error if given a 2D input.
- Then while trying with other inputs one of the music samples where so much higher in volume than the music sample so I added the volume control stage to make sure any given samples are of the same volume before processing.

- Then I padded the shorter signal with zeros so that the 2 samples are of the same length.
- Then I initialized the system parameters:
    1. Frame length M: which will be used to window the signal afterwards and I kept in mind that this needs to be a power of 2 number as this helps the FFT done in the Xcorr function behind the scenes to be optimized and faster.
    2. Hop size R: which corresponds to the jump between one audio chunk and the next so the smaller this is the move overlap between the samples and the smoother the transition and high the temporal resolution
    3. 2 LPC orders (p_speech and p_music): these control how detailed are the filters you are creating and their resolution, the higher the number the more peaks you detect and the more accurate you follow the shape of the signal fed into the LPC algorithm.
- Then I started initializing the variables related to the OLA algorithm:
    1. the tail related to the FIR Whitening filter (tail_FIR): this tail is exactly equal to the number of samples of the FIR filter which is equal to the order of the LPC which we used to get the filter coefficients (p_music).
    2. the tail related to the IIR Shaping filter (tail_IIR): this tail is not equal to the LPC order which we used to get the filter coefficients as these coefficients are used to perform and IIR filtering so they are added to the denominator in the filter function and so this results in the output being feedback to the input and so its tail doesn't decay to zero after p_speech samples which is equal to the LPC order but instead it decays until forever so it better to give it more space to completely decay until its basically not affecting the signal and that why we set it to the same hop size R.

  then we use these tails to pad the music frame inside the loop and also output buffer with zeros to prevent the aliasing effect resulting from the cyclic convolution following the rule L = M + N - 1.
- Then I initialize and output buffer which has zeros equal to the longest of the to inputs plus the total of the 2 tails,

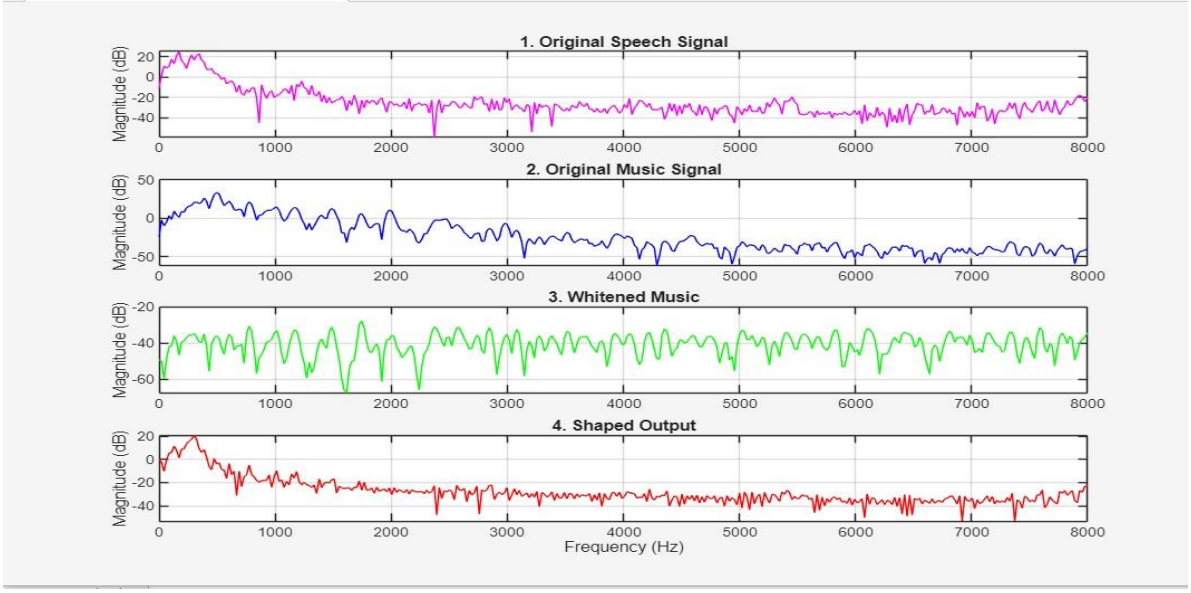
- Then before the loop I initialize a hamming window with M length (frame length); I chose a hamming window as it has the lowest first side lobe compared to the main lobe and so you get a better peak isolation and also it fades out to 0.08 at the first and last samples which is not that high as well compared to Hanning for example.
- Secondly, I went into the loop where I initialize the loop limits and the hop size R between each loop run, then for each loop I extract a window from the speech signal and the music signal, then I pad the extracted music window with the total tail to prevent aliasing as its is the signal that will be filtered by the FIR filter then the IIR.
- Then I used the unpadded music frame to get the LPC filter coefficients by applying the xcorr and Levinson-Durbin recursion frame work, these coefficients where then used to filter the padded music frame by the Whitening FIR filter using the filter function (getting the residual) , this helps to flatten the music signal so that it is ready to follow the shape of the vocal shaping filter instead of it having its own shape while it retain the pitch and harmonics (it is like an anti-Eq)
- Then I do the same frame work to the extracted speech frame to get its LPC coefficients which I then use for applying the Shaping IIR filter to the residual of the music frame and this filter captures the formants of the voice speech and applies them to the residual of the music signal resulting in the talk-box effect (synthesized frames).
- Then I append these synthesized frames to the output buffer, until the loop ends and I have the full synthesized signal in the output buffer.
- Finally, I exit the loop, normalize the output and saved to a file. So that I can play it using the sound function in the command window.

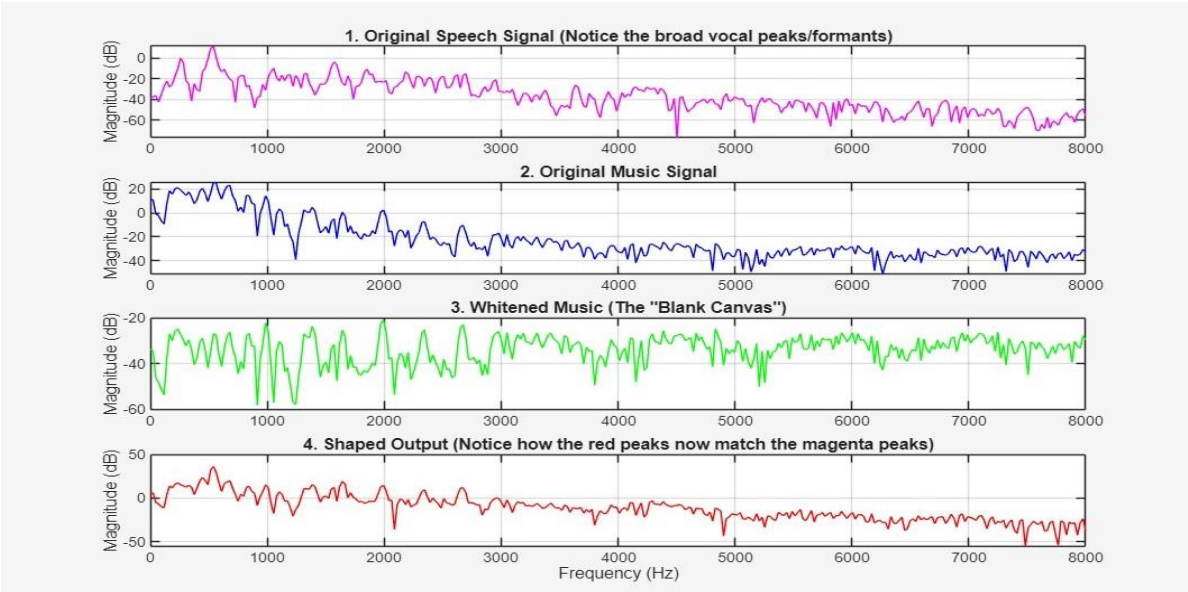
2.0 Effect Of Changing The Parameters and Results Discussion:

Now I am going to discuss the parameters I changed between changing the signals to get the best output and how they affected the process as well as the output:

- Frame Size (M): The longer the Frame size the better the frequency resolution you have, but at the same time the worse the time resolution, so if you have a long frame, you will be able to capture the harmonics and the pitch of the instrument better, but you will lose the ability to capture the fast mouth movements and consonants of the speech. So, by trying the parameters I found that the best frame sizes for the results are 1024 or 2048 which give a nice balance. Also, the frame length has to be a power of 2 to result in a faster FFT which is done in the back of the Xcorr function.
- Hop Size (R): this is defined as a fraction of the frame length which results in it defining a percentage of overlap between the current frame and the next so the lower this is the more overlap and the more smooth the transition and so the talk-box sounds less steppy and can capture the speech with a better temporal resolution.
- Speech LPC Order (p_speech): this is the LPC order for the speech which has a rule of sum that it should be between Fs/1000 and Fs/1000 + 4 and I tried to stay around this range, as the more you increase it the more peaks you can capture so if you increase it more you can start capturing the hisses of the voice which is not needed and if it is low you don't capture some of the peaks the shapes the formats, so this range I found it the best to capture the formants and create the best shaping filter.
- Music LPC Order (p_music): this is the LPC order used on the music to get the Whitening filter then apply it to the music to get the residual and the higher the order the mor the residual sounds like white noise, and through trying I found that if you put this to a low order like 40 or 50 you can still hear the instrument in the back ground and if you put it to a very high order like 128 you complete remove the instrument sound in the background and get a more of a robotic sound in the output while retaining some of the instrument harmonics and pitch, so my decision was to work in the range of 90 to 128.

- Input Signals: through experimenting with multiple input signals, I noticed that the drier the signal is the better as reverbs, delays and other effects on the input signals results in the LPC capturing bad peaks, clicks and hisses, as shown in my “synth” example, the synth had so many clicks which were captured and emphasized in the output. Also, the richer the music signal is in harmonics the better it is for the shaping filter as it makes sure there are harmonics to shape at the specific shaping points so that’s why making the residual more like a white noise is better from my opinion.
- Stage by stage analysis (Results Explanation): The two sets of graphs displayed below represent the first analysis frame extracted from two distinct audio examples. Collectively, these four plots prove that the cross-synthesis algorithm is functioning correctly by demonstrating exactly how the envelope of the human speech is imprinted onto the harmonic structure of the instrument. The algorithm's working principle is shown by the direct correlation between the original speech and shaped output graph. Specifically, the distinct vocal formants—the characteristic hills and valleys that define the spoken speech in that specific frame—have been successfully imprinted onto the final red output. Before this shaping, the blue graphs show that the raw music signals possess their own strong envelopes and instrument resonances that would have clashed with the voice. The green graphs shows that the whitening algorithm neutralizes this interference by flattening the music signal shape. Because the green signals provide a flat, neutralized canvas, the final output graphs show that the overall amplitude peaks now mirror the vocal formants. This confirms that, in both examples, the instrument's signal has successfully taken on the shape of the human mouth.





3.0 AI Contribution:

During this project, I used Google's Gemini AI to help me understand complex topics. I strictly used it to brainstorm ideas, fix code errors, and explain difficult Digital Signal Processing (DSP) concepts, such as how the different filters work and how to properly plot the audio signals. Some examples of the general prompts I used to guide my learning include: *"what does lpc orders control"* and *"why do we apply the whitening filter?"* To make sure the information was accurate and my work remained my own, I based most of the concepts and code on the lecture slides and the lectures' code examples. This allowed me to make sure I fully understood every part and topic used in my implementation.